

葱油餅パーティー (Scallion Pancake Party)

Bohan はパーティーを開催しようとしている。彼は葱油餅が好きであり、パーティー用に近くの店で葱油餅を買うことにした。この店では N 種類の異なる味の葱油餅を販売しており、 0 から $N - 1$ までの番号が付けられている。Bohan は各種類の葱油餅を N 個ずつ、合計で N^2 個の葱油餅を買うことにした。葱油餅はそれぞれ、同じ大きさの K 切れに分けられてから個別の袋にひとまとめに入れられる。袋には 0 から $N^2 - 1$ までの番号が付けられる。袋 i に入れられる葱油餅の味を $F[i]$ で表すことにする。

パーティーの中で、Bohan は参加者を全員集めてゲームに招待する。ゲームは次のように進行する。

まず、Bohan は N 人の参加者を選び、 1 つの部屋にちょうど 1 人が入るように $0, 1, \dots, N - 1$ の番号が付けられた部屋に参加者を移動させる。選ばれなかった参加者はゲームが終了するまでこれらの部屋の外にいる。これら N 個の部屋はどれも全く同じ見た目であり、参加者は自分がどの部屋にいるかわからない。

次に、それぞれの部屋 i ($0 \leq i < N$) について、Bohan は部屋 i に入り、**禁じられた味**を表す整数 $P[i]$ をその部屋にいる参加者に秘密裏に伝える。また、Bohan はその参加者に、自分がどの部屋にいるのかを伝えることができ、伝えないこともできる。Bohan は $[P[0], P[1], \dots, P[N - 1]]$ が $[0, 1, \dots, N - 1]$ の順列となるようにする。

ここから N 回のラウンドが始まる。 i 回目のラウンドでは、Bohan は袋 $0, 1, \dots, N^2 - 1$ を部屋 $i - 1$ に順番に一つずつ運び入れる。部屋 $i - 1$ にいる参加者が袋 j を受け取ると、この参加者は以下の情報を得る。

- $F[j]$, つまり袋 j の中の葱油餅の味、そして
- 袋の中に何切れの葱油餅が残っているか。

次の袋を受け取る前に、部屋 $i - 1$ にいる参加者は現在の袋の中から何切れを食べるかを決めなければならない。この参加者が何切れを食べることができるかは状況によって異なる。

- もし $P[i - 1] \neq F[j]$ であるならば、この参加者はこの袋から何切れでも取り出して食べることができる。
- もし $P[i - 1] = F[j]$ ならば、この参加者は一切れも食べることが許されない。

参加者は $F[0], F[1], \dots, F[N^2 - 1]$ をあらかじめ知っているわけではないことに注意せよ。

N 回のラウンドが終了すると、部屋の外にいる参加者はこれら袋の列を与えられる。これらの袋を見ることで、参加者はそれぞれの袋について、葱油餅の味と何切れが袋に残っているかを知ることができる。この情報をもとにして、部屋の外にいる参加者は $P[0], P[1], \dots, P[N-1]$ の値を決定しなければならない。

参加者はゲームが始まる前に事前に相談することができる。また、参加者は事前に N と K の値を知ることができる。あなたの目標は、部屋の外にいる参加者が常に正しく $P[0], P[1], \dots, P[N-1]$ の値を決定できるように戦略を実装することである。

実装の詳細

あなたは3つの関数を実装する必要がある。

あなたは部屋 $0, 1, \dots, N-1$ にいる参加者のための次の2つの関数を実装する必要がある。

```
void init(int N, int K, int p, int r)
```

参加者が部屋 i にいるとしよう。

- N : 葱油餅の味の数。
- K : ゲームの前にそれぞれの葱油餅が何切れあるか。
- $p = P[i]$: 部屋 i で食べることが禁じられている味。
- もし Bohan がこの参加者に自分がどの部屋にいるかを伝えたと決めたならば、 $r = i$ である。そうでなければ、 $r = -1$ である。

```
int strategy(int b, int f, int s)
```

参加者が部屋 i にいるとしよう。

- b : 現在の袋の番号。
- $f = F[b]$: 袋 b の葱油餅の味。
- s : 袋 b の中に何切れが残っているか。
- この関数は1つの非負整数 x を返さなければならない。これは参加者が x 切れを食べると決めることを表す。
 - もし $f \neq P[i]$ であれば、 $0 \leq x \leq s$ でなければならない。
 - もし $f = P[i]$ であれば、 $x = 0$ でなければならない。

あなたは部屋の外にいる参加者のための次の関数を実装する必要がある。

```
std::vector<int> guess(int N, int K, std::vector<int> F,  
                      std::vector<int> S)
```

- N : 葱油餅の味の数.
- K : ゲームの前にそれぞれの葱油餅が何切れあるか.
- F : 長さ N^2 の配列であり, $F[i]$ は袋 i の葱油餅の味を表す.
- S : 長さ N^2 の配列であり, $S[i]$ は N 回のラウンドが終わった後に $S[i]$ 切れが袋 i に残っていることを表す.
- この関数は長さ N の配列 P を返す必要がある. $P[i]$ は部屋 i にいる参加者に伝えられた数である.

それぞれのテストケースは T 個の独立なゲームからなる.

採点プログラムは, T 個のゲームそれぞれにおいて, $N + 1$ 個のプロセスを作成する. $0 \leq i < N$ を満たす各 i に対して, プロセス i は部屋 i にいる参加者を表す. プロセス N は部屋の外にいる参加者を表す. $0 \leq i < N$ を満たす各 i について, プロセス $(i + 1)$ が開始されるのはプロセス i が終了した後である.

プロセス 0 から $N - 1$ に関して,

- `init` はちょうど 1 回呼び出される.
- `strategy` は `init` の後にちょうど N^2 回呼び出される.
 - j 回目の呼び出しにおいて, $b = j - 1$ であることが保証される.

プロセス N に関して,

- `guess` はちょうど 1 回呼び出される.

採点プログラムは複数のゲームを並行に実行することがあるが, 個々のゲームの中でのプロセスの相対的な順序は保存されることが保証される.

制約

- $1 \leq T \leq 400$.
- $2 \leq N \leq 30$.
- 1 つのテストケースにおけるすべてのゲームにわたる N^3 の和は 27 000 を超えない.
- $1 \leq K \leq N$.
- $K \in \{1, 3, N\}$.
- $[P[0], P[1], \dots, P[N - 1]]$ は $[0, 1, \dots, N - 1]$ の順列である.
- $0 \leq j < N^2$ を満たす各 j について $0 \leq F[j] < N$ である.
- $0 \leq i < N$ を満たす各 i について $[F[0], F[1], \dots, F[N^2 - 1]]$ の中に i はちょうど N 回現れる.
- ジャッジは **適応的 (adaptive) ではない**. つまり, $[P[0], P[1], \dots, P[N - 1]]$ および $[F[0], F[1], \dots, F[N^2 - 1]]$ は最初に `init` が呼び出される前に固定される.

小課題

小課題	得点	追加の制約
1	8	$K = 1, N = 2$.
2	7	$K = 1$ であり, Bohan はすべての参加者に自分がどの部屋にいるかを伝えることを選択する.
3	14	$K = 1$ であり, $0 \leq i < N$ を満たす各 i について $[F[i \cdot N], F[i \cdot N + 1], \dots, F[i \cdot N + N - 1]]$ は $[0, 1, \dots, N - 1]$ の順列である.
4	18	$K = N$.
5	21	$K = 3, N \geq 3$.
6	32	$K = 1$.

例

$T = 1$ であり, テストケース内の唯一のゲームとして $N = 2, K = 2, P = [1, 0], F = [1, 0, 0, 1]$ となっている状況を考える.

$S[i]$ で袋 i に現在何切れが残っているかを表すことにする. 最初, $S = [2, 2, 2, 2]$ である.

ゲームの開始前に, 参加者は次の戦略を決定したとする.

- 部屋の中にいる参加者は, もし現在の葱油餅を食べることができるのならば,
 - もし現在の葱油餅が自分が食べられる最初のものであれば, 残りの全てを食べる.
 - そうでなければ, 1 切れだけを食べる.
- 部屋の外にいる参加者は,
 - もし $S = [0, 0, 1, 1]$ ならば, $P = [1, 0]$ だと推定する.
 - そうでなければ, $P = [0, 1]$ だと推定する.

ゲームは次のように進行する.

まず、ジャッジは部屋 0 にいる参加者を表すプロセス 0 を開始し、`init(2, 2, 1, -1)` を呼び出す。初期化が終わったら、ジャッジは次の呼び出しを行う。

関数の呼び出し	返り値
<code>strategy(0, 1, 2)</code>	0
<code>strategy(1, 0, 2)</code>	2
<code>strategy(2, 0, 2)</code>	1
<code>strategy(3, 1, 2)</code>	0

$P[0] = F[0] = F[3] = 1$ であるから、1 番目と 4 番目の呼び出しは 0 を返さなければならない。

このプロセスの終了後（1 回目のラウンドの後）、 $S = [2, 0, 1, 2]$ となる。

次に、ジャッジは部屋 1 にいる参加者を表すプロセス 1 を開始し、`init(2, 2, 0, -1)` を呼び出す。初期化が終わったら、ジャッジは次の呼び出しを行う。

関数の呼び出し	返り値
<code>strategy(0, 1, 2)</code>	2
<code>strategy(1, 0, 0)</code>	0
<code>strategy(2, 0, 1)</code>	0
<code>strategy(3, 1, 2)</code>	1

$P[1] = F[1] = F[2] = 0$ であるから、2 番目と 3 番目の呼び出しは 0 を返さなければならない。

このプロセスの終了後（2 回目のラウンドの後）、 $S = [0, 0, 1, 1]$ となる。

最後に、ジャッジは部屋の外にいる参加者を表すプロセス 2 を開始する。ジャッジは次の呼び出しを行う。

関数の呼び出し	返り値
<code>guess(2, 2, [1, 0, 0, 1], [0, 0, 1, 1])</code>	<code>[1, 0]</code>

部屋の外にいる参加者は正しく順列を言い当てた。従って、このテストケースは正解とみなされる。

この戦略によって部屋の外にいる参加者が常に正しく順列を言い当てることが可能であるというわけではないことに注意せよ。

この問題の添付パッケージには、この例に加えてもう 1 つ別の入力例が含まれている。

採点プログラムのサンプル

採点プログラムのサンプルは、1 つのテストケースにつき 1 回のゲームのみをサポートする。

入力のフォーマット：

```
N K
P[0] P[1] ... P[N-1]
F[0] F[1] ... F[N*N-1]
reveal
```

- `reveal` は 0 または 1 である。
 - もし `reveal` が 0 であれば, Bohan がどの参加者にも自分がいる部屋の番号を伝えないように採点プログラムのサンプルは行動する。
 - もし `reveal` が 1 であれば, Bohan がどの参加者にも自分がいる部屋の番号を伝えるように採点プログラムのサンプルは行動する。

もし `guess` によって返された配列が $[P[0], P[1], \dots, P[N - 1]]$ と一致すれば、採点プログラムのサンプルは

```
Accepted
```

と出力する。

加えて、採点プログラムのサンプルはデバッグのために、発生した関数の呼び出しを出力する。